



Deep Learning Applications Assignment 3

- Aditya Ved



Problem Statement:

Title :

Realtime Airborne Object Detection and Tracking System.

Goal:

Aeroplanes, birds, drones, and helicopters share airspace and often look alike at distance. Detecting, classifying, and tracking these four classes in real time on consumer hardware — with stable per-object identity across frames is the goal of this project.



Dataset

- Dataset link: <https://data.mendeley.com/datasets/cd5z895tr2/1>

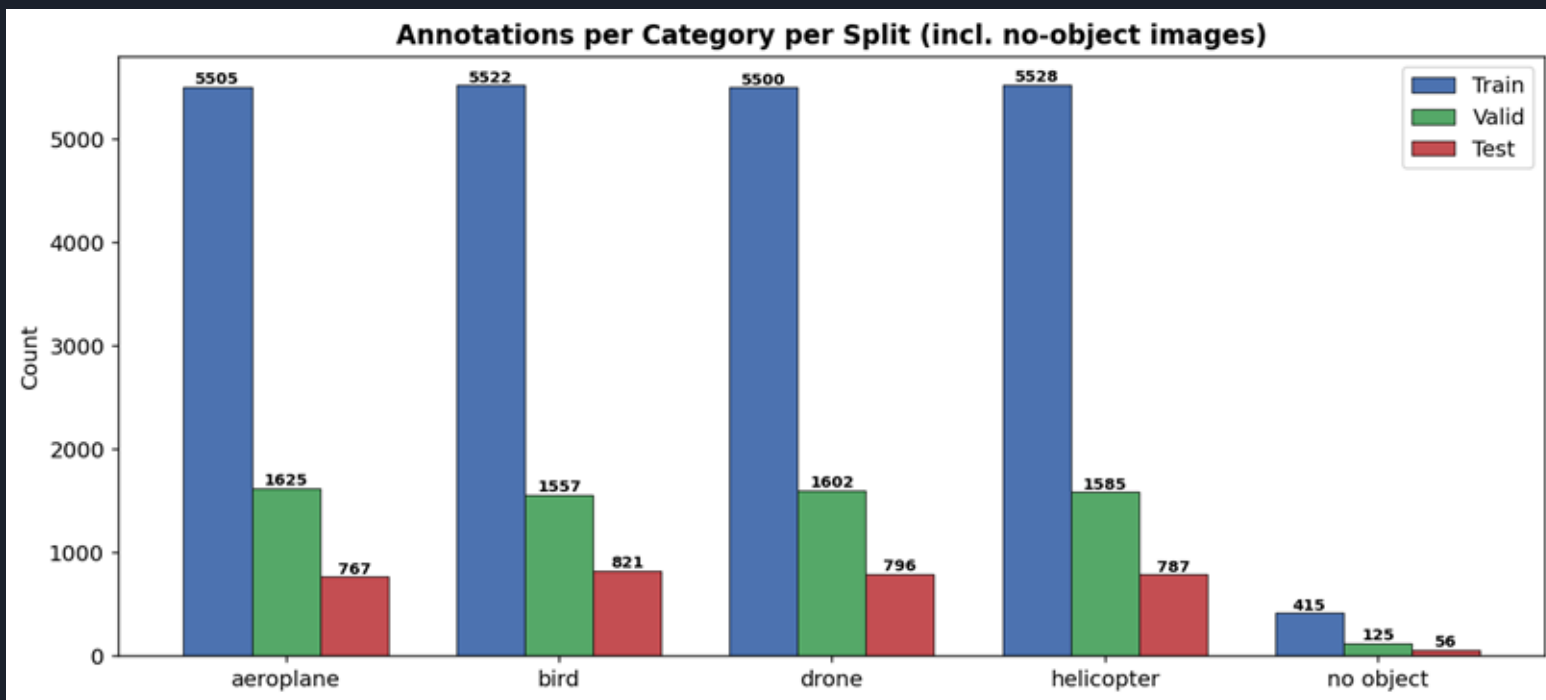
- Dataset Format:

Two main folders : images and labels. Images are given names say <file_name>.jpg and their corresponding labels (yolo v-8 onwards compatible format) are named <file_name>.txt. Basically, same name different file extension.

- Dataset Split -> Training Data : 15758 images, Validation Data : 4514 images, Testing Data : 2241 images

Dataset

Dataset is balanced as annotations are balanced in all 3 splits





Model Selection and Fine-Tuning Strategy

YOLO 26 becomes the most practical choice because it was specifically designed for Object Detection in Real time (millisecond latency) on general consumer hardware. (Details explained in Assignment 2)

YOLO 26's architecture is split into three main components:

1. Backbone
2. Neck
3. Head

We Fine-tune on the entire YOLOv26 Model (weights of all 3 components are updated during training) since we have more than 1000 annotations per class & our use case is of completely different domain (different object types, sizes, lighting conditions, and contexts).



Visualizing Training

Please refer **visualize_training.ipynb** which covers the following visualizations + observations:

1. Train vs Validation Box Loss
2. Train vs Validation Classification Loss
3. Train vs Validation DFL Loss
4. mAP@50 over Epochs
5. mAP@50 vs mAP@50-95 over Epochs — both curves on one chart
6. Precision vs Recall over Epochs



Testing Scenarios

So we tested the model on 2 types of data:

- Image files
- Video files.

Image files are already present in the Airborne Object Detection Dataset and the fine tuned model performs great on the Test split of the dataset.

To push the model to real unfamiliar zone, we then take random videos of objects flying in air and test our fine tuned model on those videos. Model performs generally well on these videos although it is not able to detect an object in certain lighting conditions and distances (basically sizes of objects in images) which hints towards the weakness of dataset as the dataset may not have covered those cases due to which the model could not learn.

Practical Understanding from the video results:

If this model has to be seriously deployed into production, then a lot of labelled image data is needed from the actual camera (in that position from where the system will get deployed). The current fine tuned model should be taken as starting point and then be fine tuned again on environment specific image data. Collecting data in different lighting conditions, different weather conditions, different distances from camera etc adds to actual diversity in the dataset which is necessary for getting a robust model.



Feature of Real Time Airborne Object Tracking in Videos

- Persistent Object Tracking across frames using Id number
- Per frame Class Count Panel (to count no of objects of each class present in frame)
- Relative Depth Increase or Decrease
- Rate of change of bounding box area “s%”
- Rough Distance Metric (Based on Assumptions)

Example

Eg : Aeroplane with ID = 1, model 86% confident that it's an aeroplane, object coming towards the camera (^), bounding box area increasing at 150% per second, plane at around 73 meters depth

In this frame:	
■ aeroplane	1
■ bird	0
■ drone	0
■ helicopter	0

id:1 aeroplane 0.86

81 ~ +150%/s ~73m





Understanding Evaluation Metrics (Loss Metrics)

1. `box_loss` — How wrong the predicted bounding box coordinates are vs. ground truth. Measures location accuracy.
2. `cls_loss` — How wrong the class predictions are (e.g., confusing a bird for a drone). Measures classification accuracy.
3. `dfl_loss` — Distribution Focal Loss trains the model to predict box edges as confident probability distributions instead of single point estimates, leading to tighter, more reliable bounding boxes.

Loss	What it measures	Question it answers
<code>box_loss</code>	Geometric similarity of predicted vs true box	"Is the box in roughly the right place with the right shape?"
<code>dfl_loss</code>	Sharpness/correctness of the distribution over edge positions	"Is the model confident and correct about where each edge lies?"



Understanding Evaluation Metrics (Performance Metrics)

- Precision — Of all boxes the model predicted, what fraction were actually correct? (Low false positives)
- Recall — Of all real objects, what fraction did the model find? (Low missed detections)
- mAP (Mean Average Precision)
 1. mAP50 — Precision averaged across all classes, where a prediction counts as correct if it overlaps $\geq 50\%$ with ground truth ($\text{IoU} \geq 0.5$). The primary "scoreboard" metric.
 2. mAP50-95 — Same but averaged across stricter IoU thresholds (0.5, 0.55, ... 0.95). Rewards tighter, more precise boxes. Harder to score high on.



Visualizing Results (Test Split of Image Dataset)

Overall Performance :

```
=====
TEST-SET PERFORMANCE — fine-tuned YOLO
=====
```

```
mAP@50      : 0.9740
```

```
mAP@50-95   : 0.7152
```

```
Precision (P) : 0.9559
```

```
Recall (R)   : 0.9413
=====
```



Visualizing Results (Test Split of Image Dataset)

Class Wise Performance :

Class	P	R	mAP50	mAP50-95

aeroplane	0.9601	0.9557	0.9829	0.7585
bird	0.9720	0.9549	0.9818	0.7611
drone	0.9470	0.9045	0.9546	0.6609
helicopter	0.9444	0.9501	0.9767	0.6802

Insights : Aeroplane and bird had the best mAP50-95 since the original Yolo 26 model was trained on aeroplane and bird as one of the 80 COCO classes. Whereas drone and helicopter had the lowest mAP50-95 as the model had to learn them from scratch using only our dataset.



Quick Notes

- No comparison with Baseline Yolo 26:

Comparison with baseline Yolo 26 model is not possible since “drones” and “helicopter” classes are not part of the 80 COCO classes with the Yolo model was trained on.

- Training Hardware and Training Time:

Yolo 26 model fine tuned on Apple M4 Max (128GB RAM) for 100 epochs (approx 10 minutes per epoch)



Important Links

Github Code : <https://github.com/AdityaVed95/Realtime-Airborne-Object-Detection-and-Tracking>

Files Description : <https://github.com/AdityaVed95/Realtime-Airborne-Object-Detection-and-Tracking/blob/main/README.md>



AI Use Statement

I wrote several comments for my code. I used AI to help me improve and polish my comments and make it more detailed so that new person reading it can more easily understand my comments and code.

Thus overall I use AI to polish my english and make my assignment look more neater and presentable and easily interpretable to a new person reading my assignment (or me referring it back after some time)



Thank You